Code Review

김영은

코드 흐름

1. 이미지 패치 단위로 잘라서 트랜스포머에 넣음

2. 트랜스포머에 넣기 전에 Soft-UMAP으로 특징을 정돈

3. 어텐션 풀링으로 중요한 패치만 강조해서 최종 분류

4. 단순히 정확도만 보지 않고, 임베딩 품질을 trustworthiness / continuity로 수치화,

   TSNE/어텐션 맵으로 시각화

trustworthiness / continuity

trustworthiness(X, X_emb) / continuity(X, X_emb)

: 고차원 공간 X와 저차원(또는 변환 후) 공간 X_emb의

국소 이웃 구조가 얼마나 잘 유지되었는가를 계산

SoftUMAPLayer

입력 x: [배치, 패치개수, 임베딩차원] 형태의 패치 임베딩

self.proj: 선형 레이어로 차원 축소 (예: 256 → 128)

dist: z끼리의 거리 행렬 (패치들 사이 거리)

weights = softmax(-dist / tau): 가까운 패치일수록 높은 가중치

z_smooth = weights @ z: 유사한 패치들끼리 정보를 서로 부드럽게 섞은 결과

z_

z_smooth : 로컬 구조를 반영해 부드럽게 정렬된 패치 임베딩

z (z_before) : 정렬 전 원본 패치 임베딩

흐름 - 데이터 준비

- MNIST불러오기

- 텐서로 변환, 정규화, 배치로더 만들기

흐름 - ManifoldAwareTransformer 생성

- 이미지 → 패치 분할
- 패치마다 임베딩 (Linear+ReLU+Linear)
- Soft-UMAP Layer로 로컬 구조 기반 스무딩 (매니폴드 인식)
- Transformer로 패치 간 상호관계 학습
- Attention Pooling으로 전역 대표 벡터 추출
- Linear로 클래스 예

흐름 - 학습 루프

- AdamW로 최적화

- label smoothing(가 있으나, 로컬 torch 버전에 따라 에러 가능)

- 매 epoch마다 test 정확도 출력, 곡선 저장

흐름 - 학습 후 분석

- TSNE로 z_before vs z_after 시각화

- trustworthiness / continuity로 임베딩 품질 수치화

- 어텐션 맵으로 “모델이 어디를 보고 판단했는가” 시각화

기존 CNN/ViT와의 차이

매니폴드 보존 (Soft-UMAP):
학습 가능한 신경망 레이어 안으로 끌어옴

신뢰성 측정 (trustworthiness, continuity):
수치로 매니폴드 품질을 확인
t-SNE 그림만 보여줄 때보다 객관적

AttentionPooling 시각화:
분류기의 내부 판단 근거를 시각적으로 설명할 수 있음

Adjustment - (1)

trustworthiness / continuity에서 n=1일 때 0으로 나누는 문제 해결

TSNE perplexity를 min(10, n_samples-1)로 제한

(TSNE는 perplexity ≥ 1, 그리고 perplexity < 샘플 수여야 하므로 실습 중단 방지)

SoftUMAPLayer에서 tau를 self.tau로 고정 저장(하이퍼파라미터 바꾸기 쉬움)

Adjustment - (1)

Transformer depth를 8→4, FFN 확장비를 8x→4x로 줄여서 학습 시간을 줄임

(단, epoch 50으로 진행)

Soft-UMAP의 tau를 0.1로 낮춤

패치 간 거리 차이를 더 강하게 강조해서, 가까운 패치끼리 더 강하게 섞이도록 함

매니폴드 평활화 효과를 더 뚜렷하게해서 전 대비 후 분포가 얼마나 달라졌는지 잘 보임

Adjustment - (1)

Transformer depth를 8→4, FFN 확장비를 8x→4x로 줄여서 학습 시간을 줄임

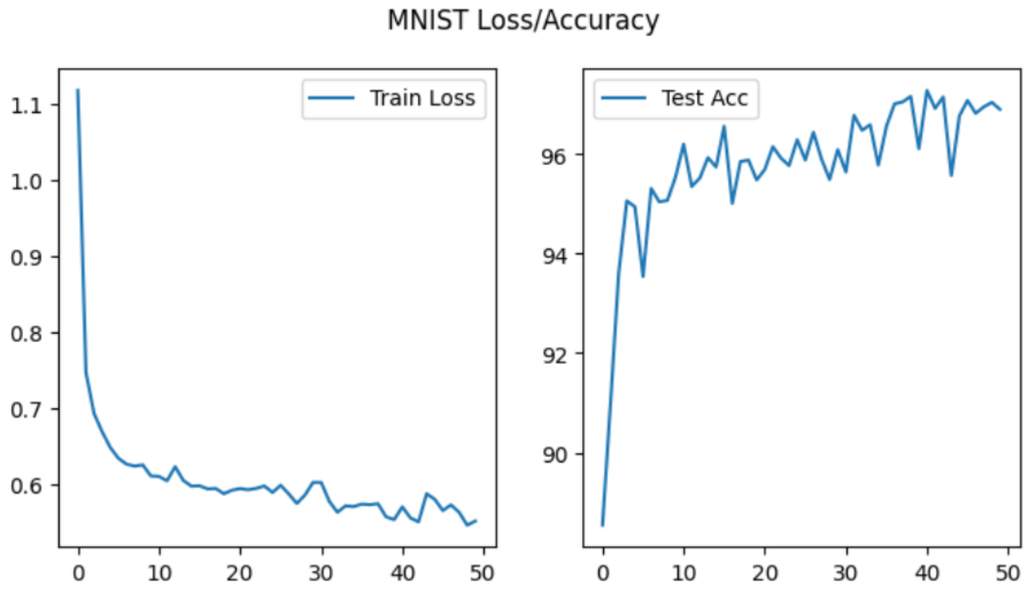
(단, epoch 50으로 진행)

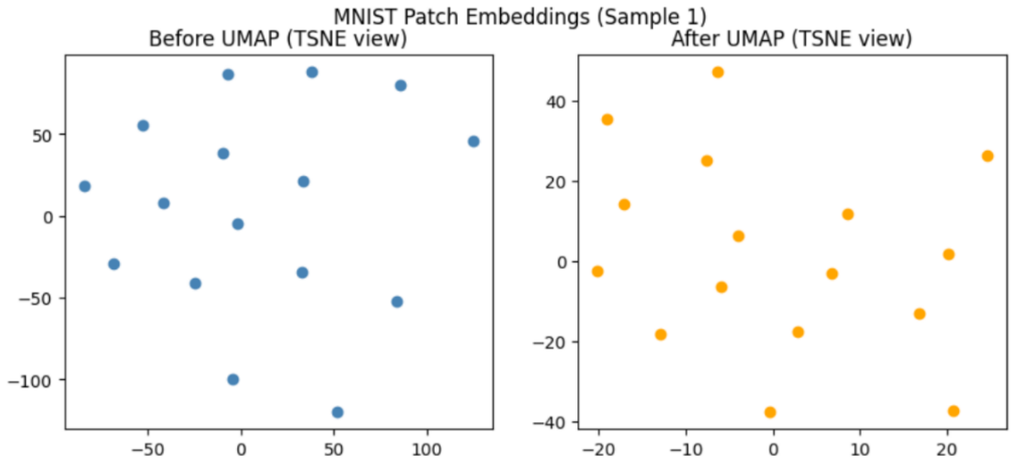
Soft-UMAP의 tau를 0.1로 낮춤

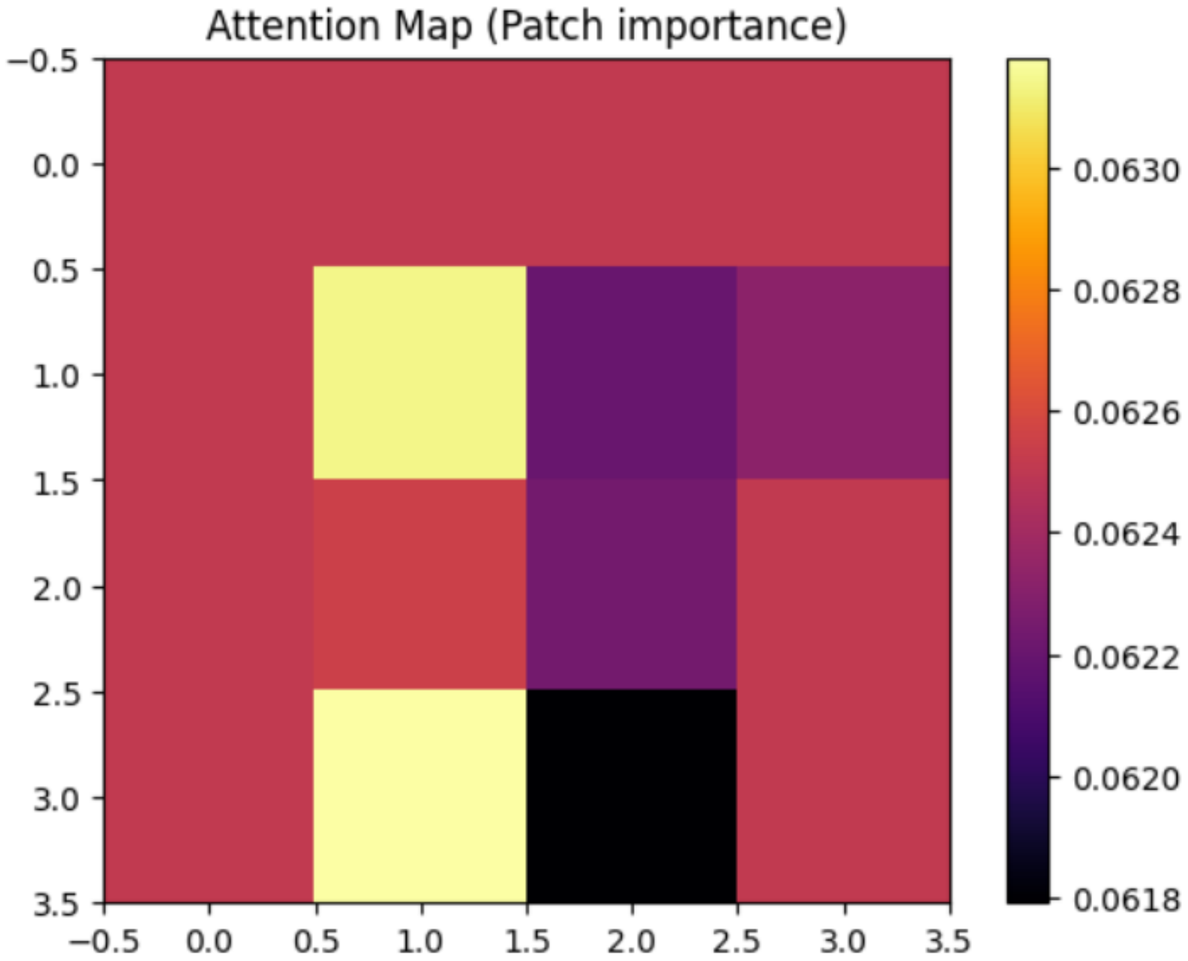
패치 간 거리 차이를 더 강하게 강조해서, 가까운 패치끼리 더 강하게 섞이도록 함

매니폴드 평활화 효과를 더 뚜렷하게해서 전 대비 후 분포가 얼마나 달라졌는지 잘 보임

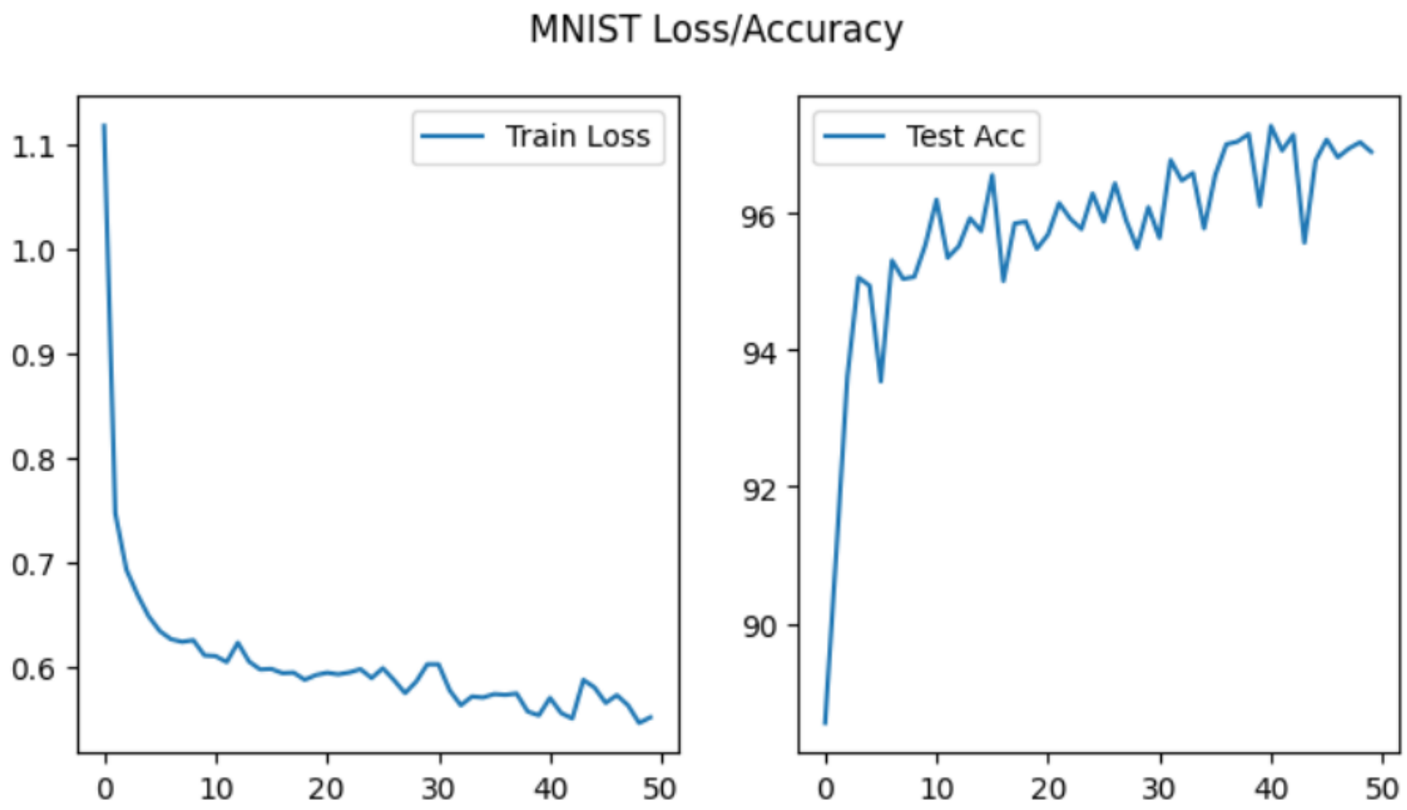
Adjustment output- (1)





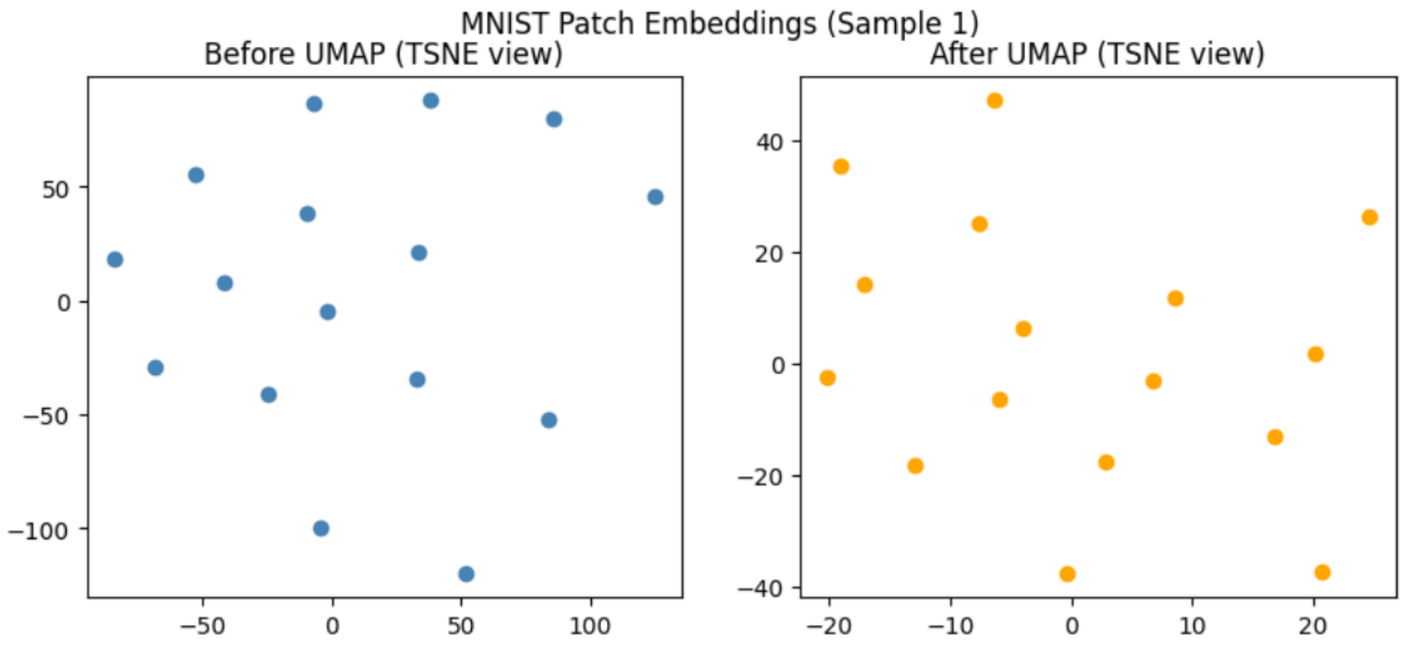


Adjustment output- (1)



Soft-UMAP + 경량 Transformer 구조가 MNIST에서는 과적합 없이 안정적으로 수렴

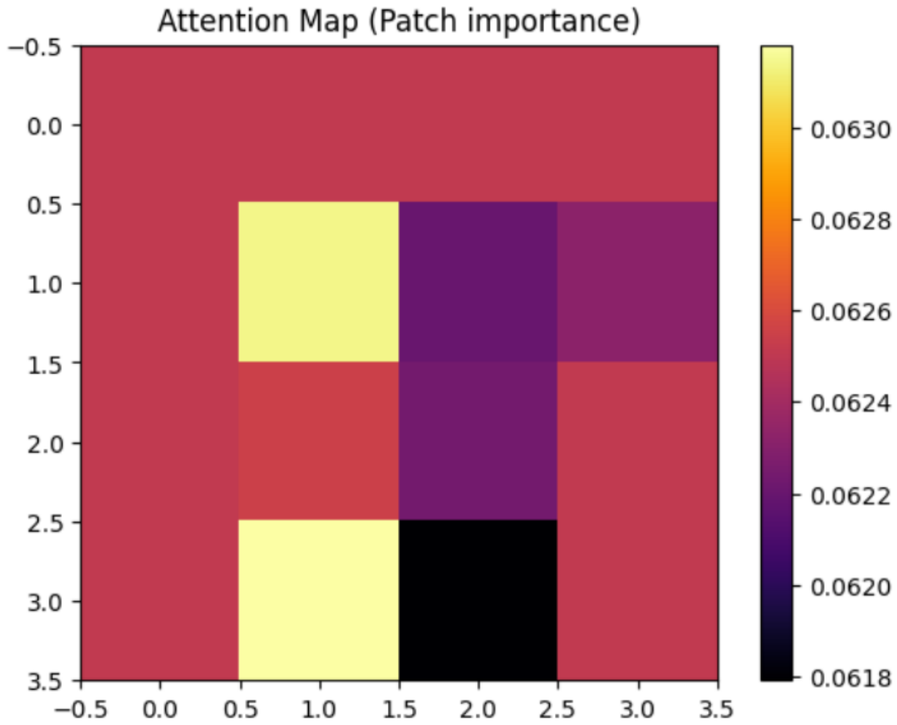
Adjustment output- (1)



국소 매니폴드 정렬 효과

TSNE 공간에서 점들이 유사 패치끼리 모이는 경향 확인 (근데 눈에 확실히 띄지는 않을지도)

Adjustment output- (1)



일부 패치에만 극단적으로 집중하는 대신 주변 패치까지 넓게 참고하는 안정적 패턴

Adjustment - (2)

정규화 강화

Attention/FFN에서 co-adaptation(특정 feature만 의존)을 줄임

특히 8-layer, ff_mult=8처럼 큰 모델에서 과적합 억제에 도움

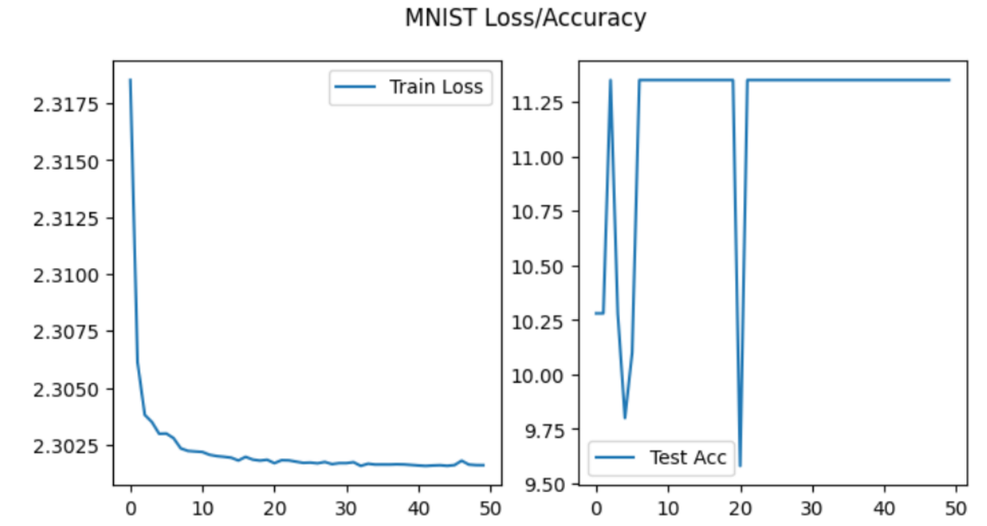
실험적으로 test accuracy 안정화, variance 감소 기대!

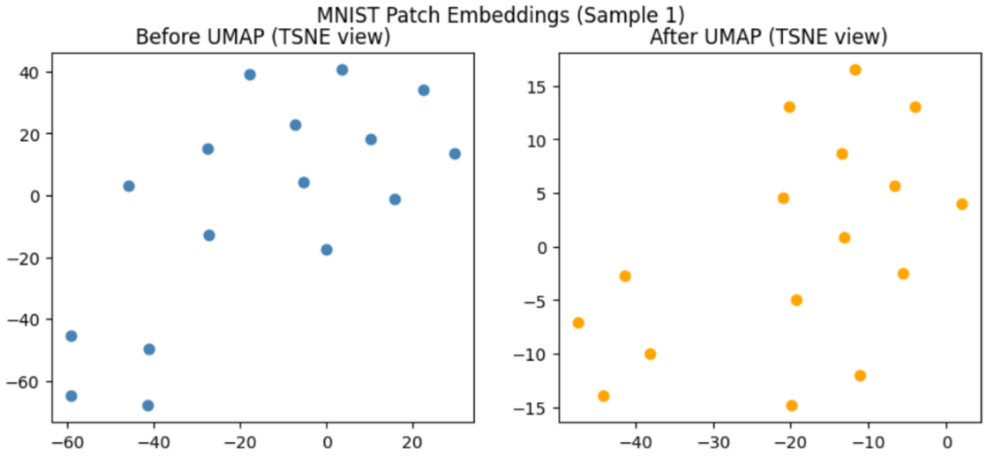
Adjustment - (2)

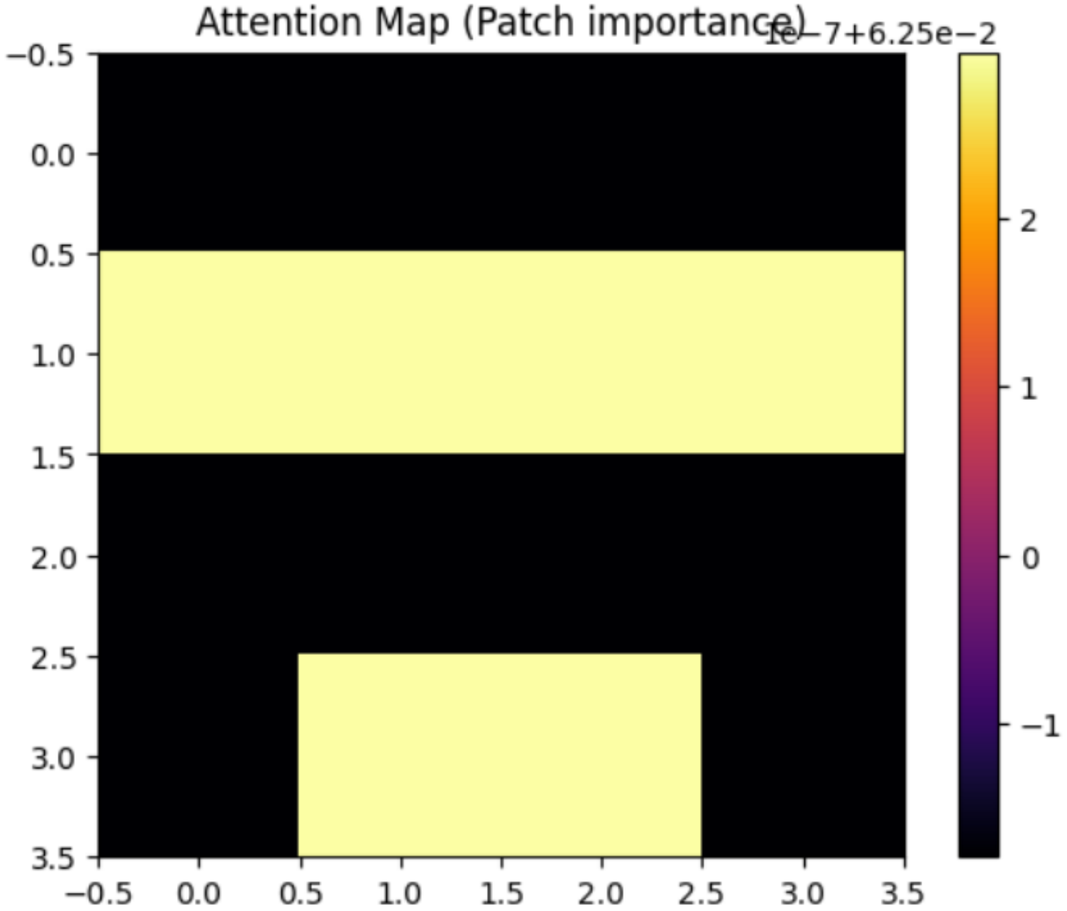
z_smooth (이웃 스무딩된 표현)과 z_before (원본 패치 표현)를 더해서 사용

: 스무딩된 표현만 쓰면 너무 평활해져서 세부 정보가 사라질 수 있기 때문

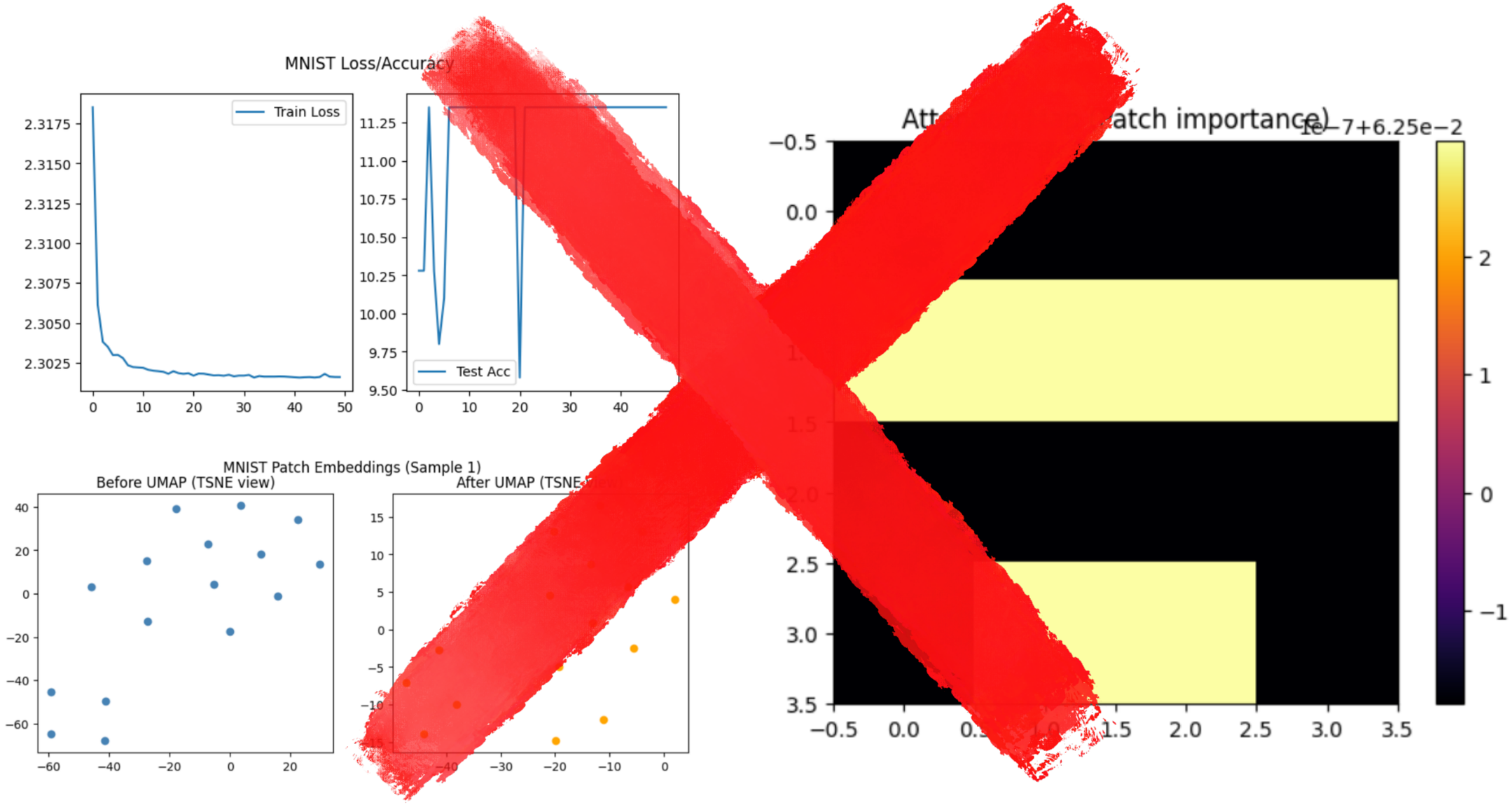
Adjustment output- (2)







Adjustment output- (2)



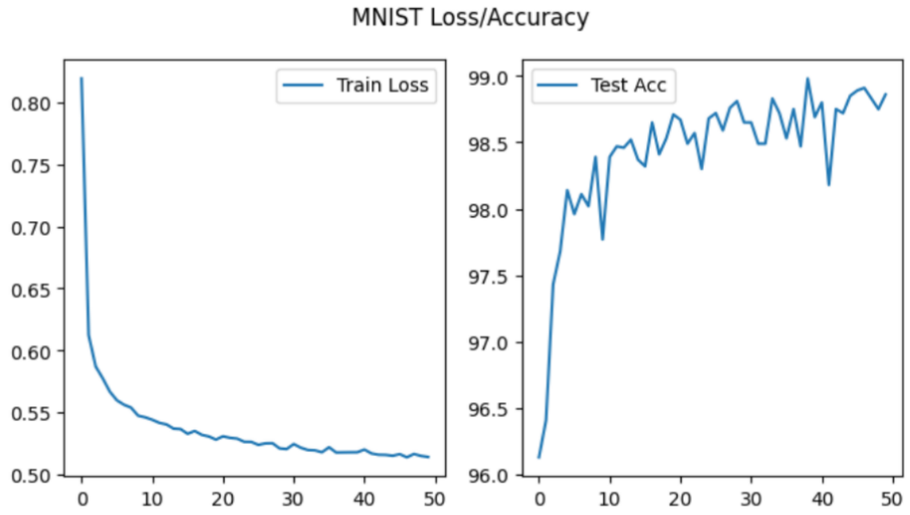
Adjustment - (3)

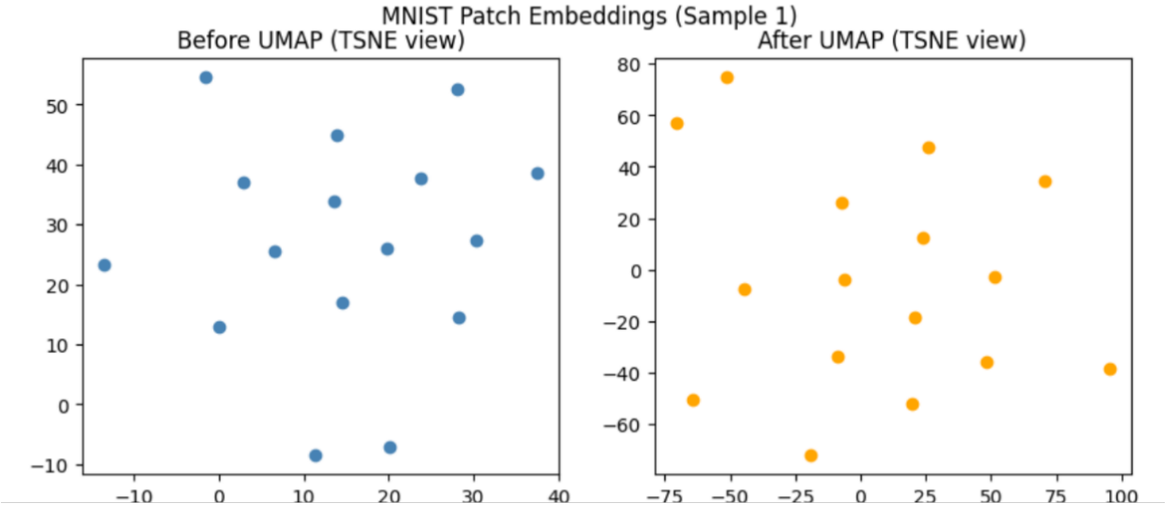
이미지를 직접 unfold 해서 패치 자르기

잘린 패치를 Linear → ReLU → Linear로 임베딩

ViT이지만 한번 도전!

Adjustment output- (3)



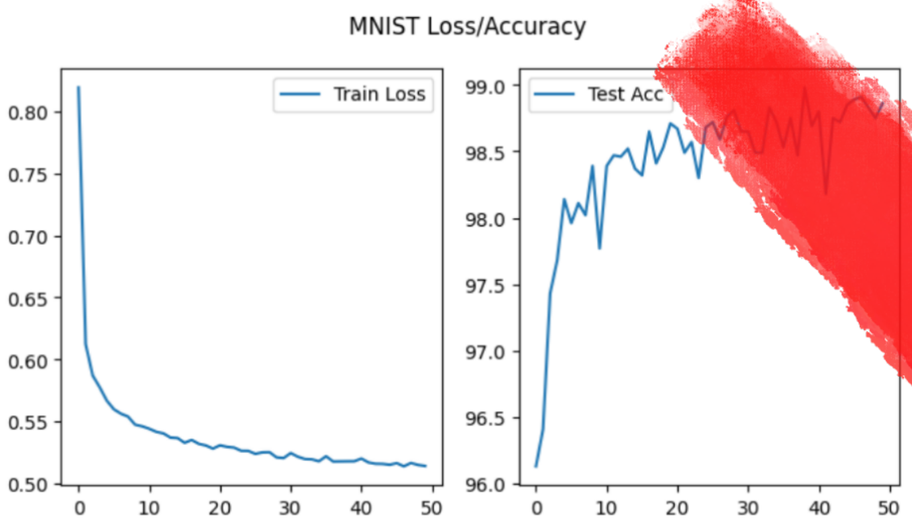


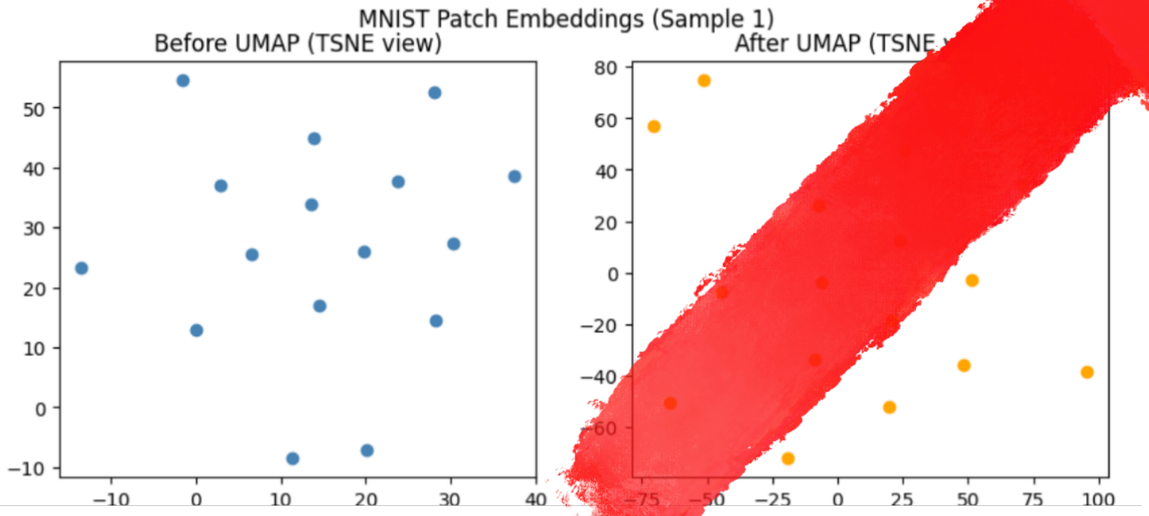
ViT 구조로 바꾸면서 학습 곡선이 훨씬 매끄럽고,

수렴 속도도 빠르며,

최종 정확도는 MNIST 상한선(≈99 %)에 근접

Adjustment output- (3)





ViT 구조로 바꾸면서 학습 곡선이 훨씬 매끄럽고,

수렴 속도도 빠르며,

최종 정확도는 MNIST 상한선(≈99 %)에 근접

그럼 논문을 왜 씀?

Adjustment - (4)

이전 : AdamW만 사용, 학습률 고정

이번 : CosineAnnealingLR(opt, T_max=epochs, eta_min=1e-6) 추가

학습률을 초반에는 크게 유지하고, 후반으로 갈수록 부드럽게 낮춰 fine-tuning

Epoch마다 scheduler.step()으로 업데이트, lr_history를 기록/플롯까지

Adjustment - (4)

이전 버전: Conv patch embedding → Soft-UMAP 바로 적용.

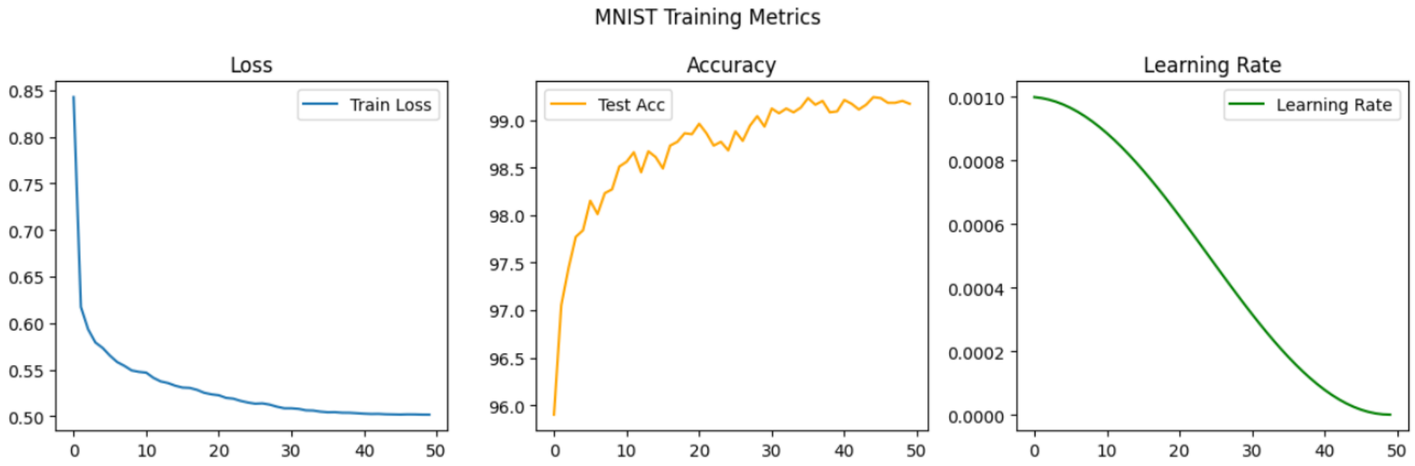
이번 버전: Conv patch embedding → LayerNorm(embed_dim) → Soft-UMAP.

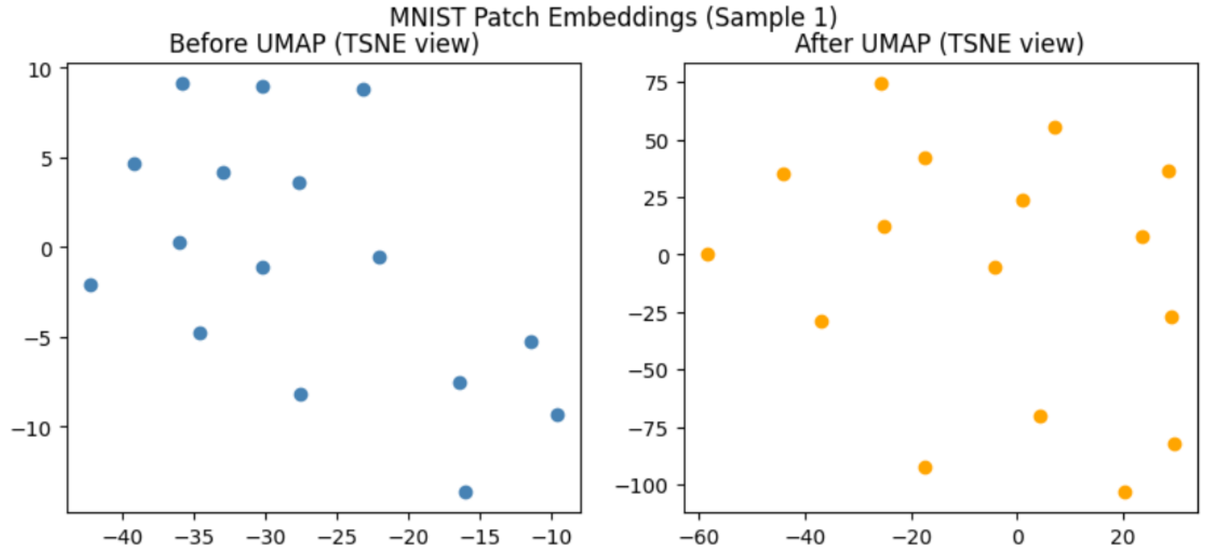
Adjustment - (4)

Soft-UMAP이 Collapse(=모든 토큰이 거의 같은 벡터)로 가는 것도 막아주고,

patch 간 거리가 이상하게 커져서 weight가 거의 one-hot이 되는 것도 막음!

Adjustment output- (4)





Adjustment output- (4)

Train Loss:
0.85 → 0.50까지 완만하게 수렴하며, 진동이나 급락이 없음

Test Accuracy:
초반 96%에서 시작해 99%에 근접한 후 완만히 plateau.

Learning Rate 그래프:
완벽한 코사인 곡선 형태로 1e-3 → 1e-6까지 감소.
→ 후반 학습에서 파라미터 진동이 억제되고, 미세 조정 단계가 제대로 수행

Adjustment output- (4)

Conv Patch Embedding ✔

Pre-UMAP LayerNorm ✔

Soft-UMAP Layer (τ=0.1) ✔

Residual Sum (z_smooth+z_before) ✔

CLS Token + Pos Embedding ✔

Pre-Norm Transformer (8L) ✔

CosineAnnealingLR ✔

AdamW + Label smoothing ✔

Code Review